Name: Bestha Akshaya Course code: CSA0611

Regno:192424336 Course name: DAA

Slot C

LAB EXPERIMENTS

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1:

Input: words = ["abc","car","ada","racecar","cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada".

Note that "racecar" is also palindromic, but it is not the first.

Code:

```
words=["abc","car","ada","racecar","cool"]
for i in words:
    if i==i[::-1]:
        print(i)
```

OUTPUT:

2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2].

Example 1:

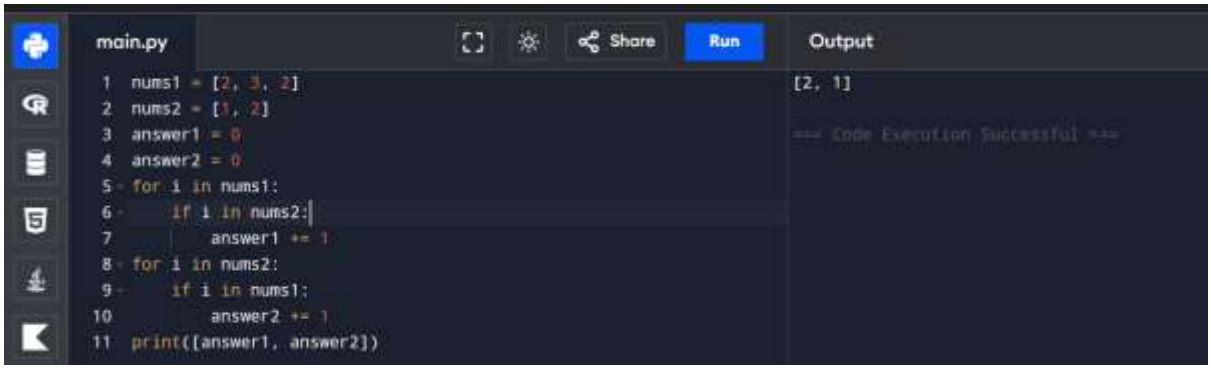
Input: nums1 = [2,3,2], nums2 = [1,2]

Output: [2,1]

Code:

```
nums1 = [2, 3, 2]
nums2 = [1, 2]
answer1 = 0
answer2 = 0
for i in nums1:
    if i in nums2:
        answer1 += 1
for i in nums2:
    if i in nums1:
        answer2 += 1
print([answer1, answer2])
```

OUTPUT:



3. You are given a 0-indexed integer array nums. The distinct count of a subarray of nums is defined as: Let nums[i..j] be a subarray of nums consisting of all the indices from i to j such that 0 <= i <= j < nums.length. Then the number of distinct values in nums[i..j] is called the distinct count of nums[i..j]. Return the sum of the squares of distinct counts of all subarrays of nums. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: nums = [1,2,1]

Output: 15

Code:

```
nums = [1, 2, 1]
ans = 0
for i in range(len(nums)):
    s = set()
    for j in range(i, len(nums)):
        s.add(nums[j])
        ans += len(s) ** 2
print(ans)
```

OUTPUT:

4. Given a 0-indexed integer array nums of length n and an integer k, return the number of pairs (i, j) where 0 <= i < j < n, such that nums[i] == nums[j] and (i * j) is divisible by k.

Example 1:

Input: nums = [3,1,2,2,2,1,3], k = 2

Output: 4

Explanation:

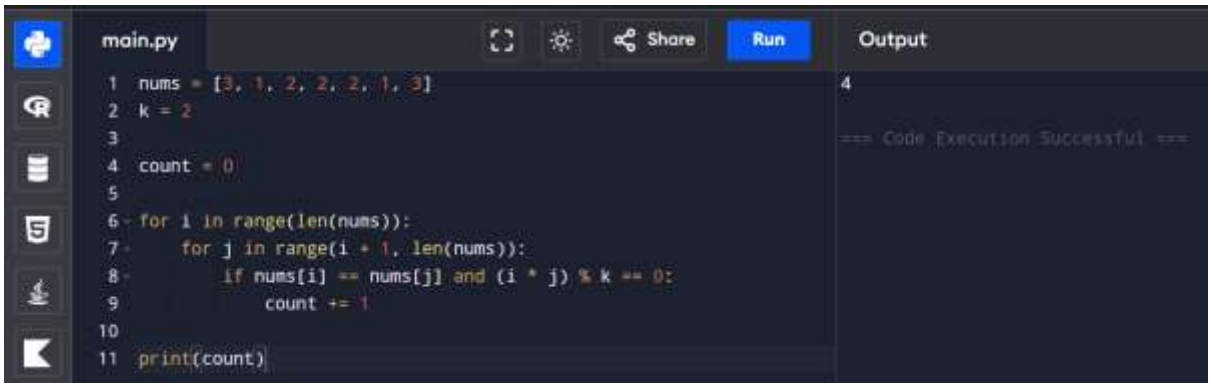
There are 4 pairs that meet all the requirements:

- nums[0] == nums[6], and 0 * 6 == 0, which is divisible by 2.
- nums[2] == nums[3], and 2 * 3 == 6, which is divisible by 2.
- nums[2] == nums[4], and 2 * 4 == 8, which is divisible by 2.
- nums[3] == nums[4], and 3 * 4 == 12, which is divisible by 2.

Code:

```
nums = [3, 1, 2, 2, 2, 1, 3]
k = 2
count = 0
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] == nums[j] and (i * j) % k == 0:
            count += 1
print(count)
```

OUTPUT:



5. Write a program FOR THE BELOW TEST CASES with least time complexity

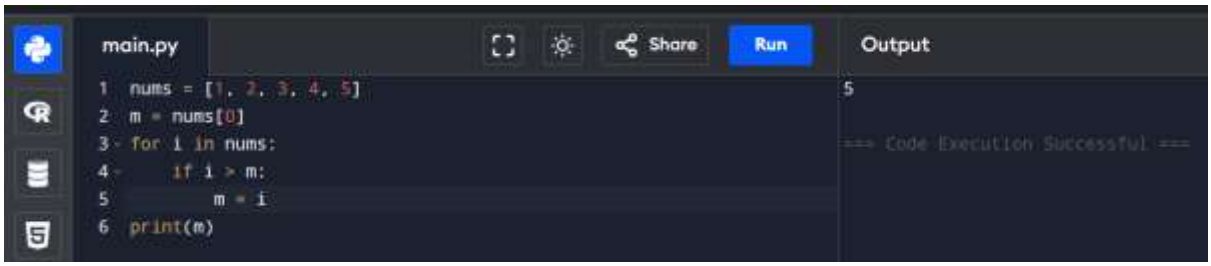
Test Cases: -

1) Input: {1, 2, 3, 4, 5}
2) Expected Output: 5

Code:

```
nums = [1, 2, 3, 4, 5]
m = nums[0]
for i in nums:
    if i > m:
        m = i
print(m)
```

OUTPUT:



6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

1. Empty List

1. Input: []

2. Expected Output: None or an appropriate message indicating that the list is empty.

Code:

```
nums = []
if len(nums) == 0:
    print("List is empty")
else:
    nums.sort()
    print(nums[-1])
```

OUTPUT:

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

· Input: [3, 7, 3, 5, 2, 5, 9, 2]

· Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Code:

```
nums = [3, 7, 3, 5, 2, 5, 9, 2]
unique = []
for i in nums:
    if i not in unique:
        unique.append(i)
print(unique)
```

OUTPUT:

8. Sort an array of integers using the bubble sort technique. Analyse its time complexity using Big-O notation. Write the code

Code:

```
nums = [5, 3, 8, 4, 2]
for i in range(len(nums)):
    for j in range(len(nums)-1-i):
        if nums[j] > nums[j+1]:
            nums[j], nums[j+1] = nums[j+1], nums[j]
print(nums)
```

OUTPUT:

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Test Case:

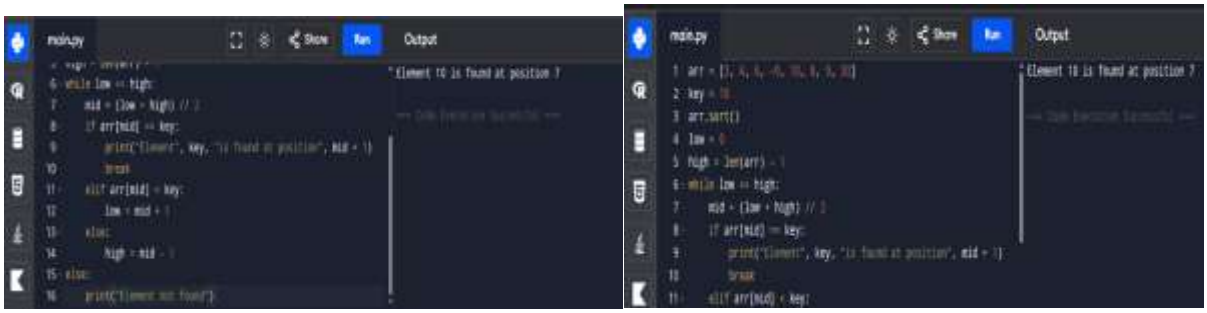
Example X={ 3,4,6,-9,10,8,9,30} KEY=10

Output: Element 10 is found at position 5

Code:

```
arr = [3, 4, 6, -9, 10, 8, 9, 30]
key = 10
arr.sort()
low = 0
high = len(arr) - 1
while low <= high:
    mid = (low + high) // 2
    if arr[mid] == key:
        print("Element", key, "is found at position", mid + 1)
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
else:
    print("Element not found")
```

OUTPUT:



10. Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

Code:

```
nums = [5, 2, 3, 1]
def merge_sort(a):
    if len(a) > 1:
        mid = len(a) // 2
        left = a[:mid]
        right = a[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                a[k] = left[i]
                i += 1
            else:
                a[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            a[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            a[k] = right[j]
            j += 1
            k += 1
merge_sort(nums)
```

print(nums)

OUTPUT: